

AbeAttributes

AbeAttributes 是一个基于 Unity Editor 的可扩展 Inspector Framework。

它旨在提供一套模块化、可扩展的 Inspector 基础设施，让 Unity Inspector 不再依赖一个高度耦合的巨大编辑器，而是由多个职责明确的系统组合而成。

AbeAttributes 关注的不只是“画出 Inspector”，而是将：

- Property Model
- Attribute
- Validation
- State
- Drawing
- Group
- Runtime Debugging

组织成一个统一的 Framework。

Why AbeAttributes?

Unity 原生 Inspector 对基础开发已经足够，但随着项目规模扩大，开发者经常需要不断增加：

- 自定义 Inspector UI
- Attribute-driven Workflow
- Validation
- Conditional Visibility
- Custom Property Drawing
- Method Interaction
- Group Layout
- Runtime Debugging

如果这些功能全部直接堆进 Inspector，代码很容易逐渐变成高度耦合的 Editor Logic。

AbeAttributes 的目标是将这些职责拆开：

Property



Process



Validate



Update State



Draw

开发者只需要关注：

“这个 **Property** 应该具有什么行为？”

而 **Framework** 负责将这些行为组合起来。

Architecture

AbeAttributes 的核心结构：

AbeInspector



AbePropertyTree



AbeProperty



Processing



Drawer Resolution



Drawer



Unity Editor GUI

Property 本身保存 **Inspector** 所需要的核心信息，例如：

AbeProperty

└— **Type / Member**

└— **Property Kind**

└— **Attributes**

└— **Value Access**

└— **State**

└ **Children**

不同系统承担不同职责。

Scanner

→ 发现 **Type / Member**

Accessor

→ 读取 / 修改 **Value**

MethodInvoker

→ 调用 **Method**

Processor

→ 处理 **Property**

Validator

→ 检查 **Property / Value**

State Updater

→ 更新 **Property State**

Drawer Locator

→ 找到匹配的 Drawer

Drawer

→ 绘制 Inspector

这种结构让新的功能可以独立加入，而不需要不断修改核心 Inspector。

Extensible Inspector

AbeAttributes 的核心设计之一是 Attribute-driven Extension。

例如：

[Required]

public string PlayerName;

或者：

[ProgressBar]

public float Health;

Attribute 不需要直接承担整个 Inspector 的职责。

不同系统可以分别处理：

Attribute

└─ Validation

└─ State

└─ Drawing

这使 Attribute 更像是：

Property Behavior 的声明方式

而不是一个巨大的功能实现类。

Validation

AbeAttributes 将 Property Validation 与 GUI Drawing 分离。

当前包含的代表性 Validation Features：

MinValue

MaxValue

Required

ValidateInput

例如：

[Required]

public string PlayerName;

Framework 可以在 Property Drawing 之前判断该 Value 是否满足要求。

因此：

Validation

≠

Drawing

这种职责分离也方便之后扩展更多 Validation Rules。

Property State

Property 的状态同样独立于 Drawing。

例如：

[ShowIf(nameof(IsVisible))]

public int Value;

Framework 可以根据条件更新：

Visible

Enabled

Expanded

Drawer 最终只需要读取 Property 当前 State，而不需要知道这个 State 是由什么条件产生的。

Custom Drawers

AbeAttributes 支持根据不同原因选择不同类型的 **Drawer**。

Attribute Drawer

当绘制方式由 **Attribute** 决定：

Attribute

↓

Attribute Drawer

例如：

[ProgressBar]

```
public int Health;
```

可以使用专门的 **Progress Bar GUI**。

Value Drawer

当绘制方式由 **Value Type** 决定：

Value Type

↓

Value Drawer

例如一个自定义类型：

```
public struct TimeRange
```

```
{
```

```
    public int Start;
```

```
    public int End;
```

```
}
```

可以为整个 **TimeRange** 类型提供统一的 **Inspector** 表现。

Property Kind Drawer

某些 **Property** 并不是普通 **Value**。

例如 **Method**:

Method

↓

Method Drawer

↓

Parameters

↓

Invoke

↓

Return Value

因此 **AbeAttributes** 也可以针对不同 **Property Kind** 提供专门的 **Inspector** 行为。

Groups

AbeAttributes 支持结构化 **Group**。

Group 使用明确的 **Start / End** 边界:

GroupStart

↓

Properties

↓

GroupEnd

这种方式可以表达复杂的 **Inspector Layout**，并允许 **Group** 结构与 **Property** 本身解耦。

Reflection Utilities

AbeAttributes 将 **Reflection** 访问集中到独立系统中。

Framework 可以统一处理:

Field

Property

Method

Member Path

因此其他系统不需要直接依赖大量 **Reflection** 操作。

对于常用的 **Member / Method Access**，也可以进行缓存，从而减少 **Inspector** 重绘过程中的重复开销。

Method Interaction

除了普通 **Property**，**AbeAttributes** 也支持 **Method-oriented Inspector Workflow**。

例如：

```
private bool IsReady()
{
    return true;
}
```

Method 可以被 **Framework** 解析并参与：

Condition

↓

Method Resolution

↓

Method Invocation

这让 **Property** 的状态、验证以及 **Inspector Interaction** 可以使用项目自身的逻辑。

PopToConsole

AbeAttributes 还包含一套 **Runtime Debugging** 功能：

PopToConsole

它可以让开发者观察：

- **Method Invocation**
- **Property Get**
- **Property Set**

- **Manual Debug Output**

例如：

```
[PopToConsole(  
    PopToConsoleMode.Set)]  
  
public int HP  
{  
    get => _hp;  
    set => _hp = value;  
}
```

当 Runtime 真正执行：

HP = 0;

时，开发者可以直接在 **Unity Console** 中看到对应的 **Debug Information**。

这让 **Inspector Metadata** 不再只服务于 **Editor GUI**，也可以服务于 **Runtime Debugging**。

Conditional Breakpoints

PopToConsole 还支持 **Conditional Breakpoints**。

例如：

```
[PopToConsole(  
    PopToConsoleMode.Set,  
    nameof(BreakIfInvalid))]  
  
public int HP  
{  
    get => _hp;  
    set => _hp = value;  
}  
  
private bool BreakIfInvalid(int value)
```

```
{  
    return value < 0;  
}
```

开发者可以定义：

“什么时候我真正想停下来检查这个状态？”

当条件满足时，**Unity Editor** 可以直接暂停。

这让：

Observe

↓

Condition

↓

Pause

成为 **Inspector Debug Workflow** 的一部分。

Current Capabilities

当前 **AbeAttributes** 已经包含：

Property Model

Type / Member Scanning

Accessor

Method Invoker

Attribute Resolution

Property Processing

Validation

State Updating

Custom Attribute Drawers

Value Drawers

Property Kind Drawers

Group Start / End

Lazy Property Children

Reflection Utilities

Cached Member Access

Runtime Debugging

PopToConsole

Conditional Breakpoints

Example

一个简单的 **Property**:

```
public int Health;
```

可以直接使用默认 **Inspector**。

加入 **Attribute**:

```
[Required]
```

```
public string PlayerName;
```

可以获得 **Validation**。

加入 **Drawing Attribute**:

```
[ProgressBar]
```

```
public float Health;
```

可以改变 **Inspector Presentation**。

加入 **Runtime Debugging**:

```
[PopToConsole(
```

```
PopToConsoleMode.Set])  
  
public int Health  
{  
    get => _health;  
    set => _health = value;  
}
```

同一个 Property Model 可以同时服务于：

Inspector

+

Validation

+

State

+

Custom Drawing

+

Runtime Debugging

Design Philosophy

AbeAttributes 的核心设计思想可以概括为：

One Property

↓

One Model

↓

Multiple Independent Systems

而不是：

One Property

↓

One Huge Inspector Implementation

这种架构带来的核心优势是：

Modularity

不同系统承担不同职责。

Extensibility

新的 **Attribute**、**Validator**、**State Logic** 或 **Drawer** 可以独立增加。

Reusability

同一个 **Property Model** 可以被多个系统共同使用。

Maintainability

GUI、**Validation**、**State** 与 **Runtime Debugging** 不需要全部耦合在一起。

In One Sentence

AbeAttributes 是一个面向 **Unity Editor** 的模块化 **Inspector Framework**，用 **Attribute**、**Property Model**、**Validation**、**State**、**Drawer** 与 **Runtime Debugging** 构建可扩展的 **Inspector Workflow**。

Project Status

AbeAttributes 当前已经具备完整的 **Inspector Core**、**Attribute-driven Workflow**、**Custom Drawing**、**Validation**、**State Management** 和 **Runtime Debugging** 能力。

项目未来可以继续围绕：

Inspector Workflow

+

Developer Tools

+

Runtime Debugging

+

Editor Productivity

进行扩展。